

Guia de estudo — Aula 5 e AV1

Detector de malware com Random Forest: o que o notebook faz, o que eu alterei e como explicar.

Os números deste guia vêm da execução local salva no notebook (scikit-learn 1.9.1). No Colab o limiar da aula saiu 0,46 em vez de 0,37, mas a história é a mesma.

0. A história em 1 minuto (decore esta parte)

1. **Problema:** decidir se um executável Windows (.exe/.dll) é malware **sem executá-lo**, só olhando números extraídos do arquivo (tamanhos, endereços, entropia, funções importadas).
2. **Notebook:** carrega uma base pública com 6.248 arquivos, escolhe 28 features, separa treino/validação/teste, treina um Random Forest, escolhe um limiar de alerta na validação e mede no teste. Resultado: recall de malware 0,96 e de benigno 0,83.
3. **O que eu descobri:** 18 das 28 features estão **zeradas** na base (dado ausente). O modelo, na prática, usa só tamanhos e layout do arquivo, que podem refletir **de onde o arquivo foi coletado** e não se ele é malicioso.
4. **Alteração 1 (features):** tirei as zeradas → nada mudou. Tirei também as que denunciam a origem → o modelo passou a errar muito mais os benignos. Conclusão: parte do acerto vinha do **viés de origem**.
5. **Alteração 2 (limiar):** escolhi o limiar pelo **custo dos erros** (malware liberado = 10, benigno bloqueado = 1). Malware liberado caiu de 42 para 4, mas os benignos bloqueados subiram de 20 para 98. Mostra que o limiar é uma **decisão de negócio**, não do modelo.

1. O problema de segurança

Um antivírus precisa decidir rápido se um arquivo é perigoso. Existem dois jeitos:

- **Análise dinâmica:** executar o arquivo numa sandbox e observar o comportamento. É mais confiável, porém lento e arriscado.
- **Análise estática:** ler o arquivo **sem executar** e extrair números dele. É o que o notebook usa.

Executáveis Windows seguem o formato **PE (Portable Executable)**. Ele tem cabeçalhos com metadados, **seções** (código, dados, recursos) e uma **tabela de imports** (funções do Windows que o programa vai chamar). Essas partes viram as *features* do modelo.

Segurança do laboratório: a base só tem tabelas CSV com números. Nenhum malware é baixado nem executado.

2. A base de dados

UCI Malware Static and Dynamic Features VxHeaven and VirusTotal (id 541, licença CC BY 4.0). São três CSVs:

Arquivo	Conteúdo	Amostras	Rótulo
staDynBenignLab	programas de <i>Program Files</i> do Windows 7/8	595	0 = benigno
staDynVt2955Lab	malwares do VirusTotal (2018)	2.955	1 = malware
staDynVxHeaven2698Lab	malwares do VxHeaven (coleção antiga)	2.698	1 = malware

Total: **6.248 arquivos, 90,5% malware**. Os três CSVs têm 1.085 colunas em comum.

Dois problemas que você precisa saber explicar:

- **Desbalanceamento:** só 9,5% são benignos. Um "modelo" que diz sempre "malware" acerta 90,5%. Por isso **acurácia engana** aqui.
- **Viés de origem:** cada fonte tem um rótulo só. Todo benigno veio de um lugar, todo malware veio de outros. O modelo pode aprender "isso parece um programa instalado do Windows 7" em vez de "isso é malicioso".

3. As 28 features, em português claro

Estrutura PE (11 features) — as únicas que funcionam nesta base

Feature	O que é
filesize	tamanho do arquivo
size_code	tamanho da área de código
size_init_data / size_uninit_data	tamanho das áreas de dados com valor inicial / sem valor inicial
size_of_headers	tamanho dos cabeçalhos do PE
size_of_optional_header	tamanho do "optional header" (onde ficam endereços e alinhamentos)
AddressOfEntryPoint	endereço onde a execução começa
image_base	endereço de memória onde o programa prefere ser carregado
section_alignment / file_alignment	como as seções são alinhadas na memória / no disco
number_of_sections	quantas seções o arquivo tem (zerada na base)

image_base, alinhamentos e tamanhos de cabeçalho são definidos pelo **compilador/linker**, não pelo comportamento do programa. Por isso eu os chamei de "features de origem": denunciam a ferramenta e a época em que o arquivo foi gerado.

Entropia e seções (7 features) — todas zeradas

Entropia mede o quanto os bytes parecem aleatórios (escala de 0 a 8). Entropia alta (perto de 8) sugere conteúdo **comprimido ou criptografado**, comum em malware "empacotado" (packer) que esconde o código real. Em teoria seriam features ótimas, mas nesta base estão vazias.

Imports / capacidades (10 features) — todas zeradas

Funções do Windows que o arquivo diz que vai usar:

API	Uso suspeito típico
VirtualAlloc, VirtualProtect	alocar memória e torná-la executável (desempacotar código)
WriteProcessMemory, CreateRemoteThread, OpenProcess	injeção de código em outro processo
RegSetValueExA	escrever no registro (persistência)
InternetOpenA, URLDownloadToFileA	acessar a internet / baixar outro arquivo

Uma API sozinha não prova nada (programas legítimos também usam), mas a combinação é um sinal forte. Também estão zeradas.

4. O notebook, seção por seção

Seção	O que faz	O que você diz
Bibliotecas	importa pandas, sklearn etc. e fixa <code>RANDOM_SEED = 42</code>	a seed garante que o resultado se repete
1. Baixar	baixa o zip do UCI para <code>/content</code>	só CSV, nada executável
2. Carregar	junta os 3 CSVs mantendo as colunas comuns e marca a origem	a tabela mostra que origem = rótulo
Auditoria (minha)	conta valores $\neq 0$ nas 18 colunas	só 2 em cerca de 112 mil células
3. Features	define as três famílias (28 features) e monta X e y	conjunto pequeno e explicável
4. Exploração	boxplots de tamanho, seções, entropia, imports	só o tamanho difere; o resto fica achatado em zero
5. Split	60% treino, 20% validação, 20% teste, estratificado	cada parte mantém 90,5% de malware
6. Baseline	<code>DummyClassifier</code> : sempre "malware"	acurácia alta, mas bloqueia todos os benignos
7. Random Forest	treina o modelo e avalia na validação com limiar 0,50	
8. Matriz de confusão	mostra os acertos e os erros na validação	
9. Limiar	escolhe o limiar na validação: recall $\geq 95\%$ com a maior precisão $\rightarrow$ 0,37	só olha a validação
10. Teste	aplica o limiar 0,37 no teste, uma única vez	resultado "oficial"
11. Importâncias	quais features o modelo mais usou	só estrutura; entropia e imports = 0
12. Ablção	treina com cada família separada (validação cruzada)	entropia e imports = chute
13. Limitações	viés, idade, split, rótulo, evasão	
14–15. Splunk	exporta um CSV e mostra o SPL equivalente	opcional

5. Os conceitos de Machine Learning que podem cair

Treino, validação e teste — por que três partes?

- **Treino (3.748 amostras)**: o modelo aprende aqui.
- **Validação (1.250)**: onde **eu tomo decisões** (escolher o limiar).
- **Teste (1.250)**: usado **uma vez só**, no final, para medir o resultado.

Se eu escolhesse o limiar olhando o teste, o teste deixaria de ser uma medida independente: eu estaria "colando na prova". **Estratificar** significa manter a mesma proporção de malware (90,5%) nas três partes.

Baseline (DummyClassifier)

É o modelo "burro" que sempre responde a classe mais comum. Serve como régua: se o Random Forest não superar o Dummy, ele não aprendeu nada. Aqui o Dummy tem acurácia de cerca de 90%, mas recall de benigno 0.

Random Forest

- Uma **árvore de decisão** faz perguntas em sequência ("`filesize > 200 KB`? `AddressOfEntryPoint < X`?") até chegar a uma resposta.
- Uma **floresta** são várias árvores (aqui **250**), cada uma treinada com um sorteio diferente de amostras e features. No final elas **votam**.
- `predict_proba` devolve a **fração de árvores** que votou "malware". Esse número vai de 0 a 1.
- `class_weight='balanced'`: como há poucos benignos, cada benigno pesa cerca de 9,5 vezes mais no treino. Sem isso, o modelo ignoraria a classe minoritária.

- `min_samples_leaf=2`: cada "folha" da árvore precisa ter pelo menos 2 exemplos, o que evita decorar casos isolados (overfitting).

Limiar (threshold)

O modelo dá uma probabilidade e o **limiar** decide a partir de quanto vira alerta. Se a probabilidade for $\geq$ limiar, o arquivo é tratado como malware.

- **Limiar menor** → mais arquivos marcados como malware → menos malware escapa (FN ↓), mais benignos bloqueados (FP ↑).
- **Limiar maior** → o contrário.

Não existe limiar "certo". Ele depende de quanto custa cada tipo de erro.

Matriz de confusão e os dois erros

	Previsto benigno	Previsto malware
Real benigno	TN (acerto)	FP : benigno bloqueado
Real malware	FN : malware liberado	TP (acerto)

No teste do modelo original (limiar 0,37): **TP 1.089, FN 42, FP 20, TN 99**.

Métricas

- **Recall de malware** = $TP / (TP + FN) = 1.089 / 1.131 = \mathbf{0,963}$. É a fração dos malwares que o modelo pegou.
- **Recall de benigno** = $TN / (TN + FP) = 99 / 119 = \mathbf{0,832}$. É a fração dos benignos que foram liberados corretamente.
- **Precisão de malware** = $TP / (TP + FP) = 1.089 / 1.109 = \mathbf{0,98}$. Quando o modelo diz "malware", acerta 98% das vezes.
- **F1**: média harmônica de precisão e recall. Um número só que equilibra os dois.
- **ROC-AUC**: chance de o modelo dar nota maior a um malware sorteado do que a um benigno sorteado. 0,5 = chute, 1 = perfeito. Não depende do limiar.
- **PR-AUC (average precision)**: resumo da curva precisão $\times$ recall. O **chute vale a proporção de positivos** (0,905 aqui). Por isso PR-AUC 0,905 significa "não aprendeu nada".

Por que olhar o recall de benigno? Com 90,5% de malware, as métricas da classe malware ficam altas quase de graça. O recall de benigno mostra se o modelo realmente distingue as classes. Atenção: só há 119 benignos no teste, então **cada FP a mais derruba cerca de 0,84 ponto** nesse recall.

Importância de features

Mostra o quanto cada feature ajudou as árvores a separar as classes. **Não é causalidade**: uma feature importante pode ser só um atalho (viés de coleta). Resultado: `AddressOfEntryPoint`, `size_of_headers`, `filesize`, `size_init_data` e `size_code` somam cerca de 80%. Entropia e imports = 0.

Ablação com validação cruzada

Ablação é remover partes para ver o quanto cada uma contribui. O notebook treina um modelo só com cada família, usando **validação cruzada de 5 folds**: divide treino + validação em 5 pedaços e treina 5 vezes, testando cada vez em um pedaço diferente. Depois tira a média.

Família	recall	F1	PR-AUC
Estrutura	0,941	0,962	0,997
Entropia/seções	0,600	0,570	0,905 = chute
Imports/capacidades	0,600	0,570	0,905 = chute
Todas	0,931	0,958	0,996

Só Estrutura já basta, e as outras duas famílias são iguais a chutar.

6. O achado central: as features zeradas

Na célula de auditoria:

- as 18 colunas (entropia, imports e number_of_sections) já vêm como **inteiros** no CSV, então não foi erro de conversão;
- em cerca de 112 mil células, apenas **2 são diferentes de zero**;
- **todo PE tem pelo menos 1 seção**, então number_of_sections = 0 é impossível. Esses zeros são **dado ausente** na base original, e não medição.

Consequência: o modelo decide só com tamanhos e layout, justamente as features que mais refletem **compilador, época e fonte da coleta**. Elas também são **fáceis de manipular** por um atacante (adicionar bytes de enchimento, recompilar, mudar o alinhamento), o que liga o resultado à **evasão**.

7. Alteração 1 — Dados e Features

Ideia: testar se o modelo detecta malware ou detecta **de onde o arquivo veio**.

- **1a:** remover as 18 zeradas → sobram 10 features.
- **1b:** remover as zeradas e também as 5 "de origem" (image_base, file_alignment, section_alignment, size_of_headers, size_of_optional_header) → sobram 5.

Como: mesmo split, mesma seed, mesmos hiperparâmetros. Só muda a lista de colunas. Cada variante escolhe o próprio limiar na validação, com a mesma regra da aula.

	recall malware	recall benigno	FN	FP	ROC-AUC
Original	0,963	0,832	42	20	0,973
1a	0,958	0,832	48	20	0,974
1b	0,959	0,588	46	49	0,944

Leitura:

- 1a ≈ original: as zeradas eram ruído puro. Era o que a ablação já indicava.
- 1b: o **benigno** desaba. Sem as marcas de compilador, o modelo não reconhece mais a "cara" de programa instalado do Windows 7/8.
- **Por que o recall de malware não caiu?** Porque 90% da base é malware: na dúvida, o modelo diz "malware" e acerta quase sempre. O erro aparece todo do lado dos benignos.

Frase para o professor: "Parte do desempenho original vinha de reconhecer a origem da coleta, e não comportamento malicioso. Além disso, essas features são baratas de forjar, então um atacante poderia evadir o detector."

8. Alteração 2 — Parâmetros e Decisão (limiar por custo)

Ideia: a regra da aula ("recall ≥ 95%") é arbitrária. Num antivírus os erros têm custos diferentes. Suponho que **malware liberado (FN) custa 10** e **benigno bloqueado (FP) custa 1**.

Como: para cada limiar de 0,05 a 0,95, calculo na **validação** o custo = $10 \cdot FN + 1 \cdot FP$ e escolho o limiar de menor custo. O gráfico mostra essa curva. Aplico o limiar no modelo original (sem retreinar) e meço no teste.

	limiar	FN	FP	recall benigno	custo no teste
Regra da aula	0,37	42	20	0,832	440
Limiar por custo	0,07	4	98	0,176	138

Leitura: pelo critério de custo, melhorou muito (440 → 138). Mas o limiar 0,07 bloqueia **82% dos programas legítimos**, o que inundaria o SOC de chamados. Minimizar só o custo esconde esse problema operacional.

Sensibilidade (o 10:1 é um chute meu, então testei outras razões):

FN custa ... × o FP	1	2	5	10	20
limiar ótimo	0,24	0,14	0,09	0,07	0,07

- Quanto mais caro o FN, menor o limiar: o modelo alerta mais.
- Mesmo com custos **iguais** (1:1), o ótimo (0,24) fica abaixo de 0,37, porque há muito mais malware do que benigno na base.
- **Conclusão:** a razão de custo tem que vir **do negócio e da capacidade do SOC**, não do modelo. Na prática eu combinaria custo com um teto de FP.

9. Regras que eu segui (o professor vai gostar de ouvir)

- **Não editei nenhuma célula original:** o notebook original é o "antes" e as alterações estão numa seção nova no final.
- **Mudei uma coisa por vez:** mesmo split, seed 42, 250 árvores, min_samples_leaf=2, class_weight='balanced'.
- **Todo limiar sai da validação.** O teste só aparece na função resumo, depois que tudo está decidido.
- **Única mudança fora da seção da AV1:** movi as células de auditoria para depois da definição das features. No Colab elas tinham sido rodadas fora de ordem, e numa execução de cima a baixo davam erro. Não mudei o conteúdo delas.

10. Perguntas prováveis e respostas curtas

"O que é o limiar?" O valor de probabilidade a partir do qual o arquivo vira alerta. Abaixar o limiar pega mais malware, mas bloqueia mais software legítimo.

"Por que não usar acurácia?" Porque 90,5% é malware. Um modelo que sempre diz "malware" tem 90% de acurácia e é inútil. Olho recall por classe, FN e FP.

"Por que a 1b piorou só o benigno?" Ela tirou as features que descrevem o padrão de compilação dos programas instalados. Sem elas o modelo não reconhece benigno, e como malware é maioria, ele chuta "malware" e continua acertando essa classe.

"O limiar por custo é melhor ou pior?" Melhor pelo custo que eu defini (440 → 138), mas inviável na operação (82% dos benignos bloqueados). O experimento mostra que o custo precisa ser definido com a operação e combinado com um limite de FP.

"Você usou o teste para decidir algo?" Não. Todos os limiares (0,40, 0,41, 0,07 e a sensibilidade) usam só y_valid e proba_valid.

"Por que PR-AUC de 0,905 é chute?" Porque o PR-AUC de um modelo aleatório é igual à proporção de positivos na base, que é 90,5% de malware.

"Como saber que os zeros são dado ausente e não medição?" Todo PE tem pelo menos uma seção, e number_of_sections está zerado. Além disso, as 18 colunas inteiras têm só 2 valores diferentes de zero.

"O que você faria para melhorar?" Usaria uma base mais recente e com benignos e malwares da mesma época (EMBER ou BODMAS), dividiria por data ou família em vez de aleatoriamente, recuperaria entropia e imports reais e definiria o custo junto com o SOC.

"Por que essas features são ruins contra um atacante?" Tamanho, alinhamento e endereços podem ser alterados com padding ou recompilação sem mudar o que o malware faz. Isso é **evasão**.

11. Glossário rápido

Termo	Significado
Feature	uma coluna numérica que descreve o arquivo
PE	formato de executável do Windows
Entropia	grau de "aleatoriedade" dos bytes; alta = comprimido ou criptografado
Packer	ferramenta que comprime ou esconde o código do malware
Import / API	função do Windows que o programa chama
Baseline	modelo de referência mínimo
Estratificar	manter a proporção das classes em cada parte
Overfitting	decorar o treino e ir mal em dados novos
FN	malware que passou (o erro mais perigoso)
FP	programa legítimo bloqueado (gera chamado e atrapalha o usuário)
Recall	dos casos reais de uma classe, quantos o modelo acertou
Precisão	dos alertas emitidos, quantos estavam certos
Ablação	remover partes para medir a contribuição de cada uma
Validação cruzada	treinar e testar várias vezes em pedaços diferentes e tirar a média
Viés de origem	o modelo aprende a fonte da coleta em vez do fenômeno
Evasão	o atacante altera o arquivo para escapar do detector
SOC	centro de operações de segurança, a equipe que trata os alertas
Splunk / SPL	plataforma de SIEM e sua linguagem de busca; o notebook exporta um CSV para treinar lá